Progress in Artificial Intelligence
https://doi.org/10.1007/s13748-023-00311-y

REGULAR PAPER

Automated machine learning hyperparameters tuning through meta-guided Bayesian optimization

Moncef Garouani[1,2] · **Mourad Bouneffa**[1]

Received: 13 April 2023 / Accepted: 6 December 2023
© Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract
The selection of one or more optimized Machine Learning (ML) algorithms and the configuration of significant hyperparameters are among the crucial but challenging tasks for the advanced data analytics using ML methodologies. However, it is one of the essential tasks in order to apply the ML-based solutions to deal with the real-world problems. In this regard, Bayesian Optimization (BO) is a popular method for optimizing black-box functions. But, yet it is deficient for large-scale problems because it fails to leverage the knowledge from historical applications. The major challenge in this aspect is due to the waste of function evaluations on bad design choices of ML hyperparameters. To address this issue, we propose to integrate Bayesian Optimization via Meta-Guidance. Consequently, Meta-Guided Bayesian Optimization provides means to use the knowledge from previous optimization cycles on similar tasks. This capability takes the form of pre-requisite to decide the specific parts of the input space to be evaluated next. In this regard, we intend to guide the BO with a functional ANOVA of configurations as suggested by a meta-learning process. In this paper, we demonstrate, with the help of a large collection of hyperparameters optimization benchmark, that the proposed Meta-Guided Optimization approach is about 3 times faster than the vanilla BO. Thence, it achieves a new state-of-the-art performance as proved by the experiments on 09 classification datasets.

Keywords Hyperparameters optimization · Bayesian optimization · Meta-learning · Meta-Guided Bayesian Optimization · fANOVA

1 Introduction

The selection of appropriate machine learning algorithms and the optimization of related Hyperparameters (HPs) are ubiquitous tasks in machine learning. These tasks are typically specific to the research space and real-world applications being considered [1]. The configuration of hyperparameters is an important design choice to obtain the desired performance of ML algorithms, whereas the correct configuration of hyperparameters may require significant consumption of valuable resources in terms of time and computational resources, as it requires high technical expertise in ML and mathematics along with many *ad-hoc* choices [2]. As a result, many of the recent research efforts in machine learning have been concentrated on improving methods for optimizing ML algorithms hyperparameters [1, 3–8].

One of the most effective methods for optimizing machine learning algorithms is commonly recognized to be sequential model-based optimization, also referred to as Bayesian Optimization [4]. Empirical evidence has shown that it has produced best results for hyperparameters tuning of convolutional networks compared to domain experts, consistently achieving higher scores on the CIFAR-10 benchmark through repeated testing [9]. However, Bayesian Optimization is defined as a generic optimization function framework, and like any other generic optimization method, it necessitates a significant amount of function assessments to identify optimal areas of high performance when initiated on a new optimization task.

While the same ML algorithms with the same hyperparameters space are systematically optimized for different tasks (datasets), the human experts rather often study the performance of the given algorithm on previously seen tasks, to

✉ Moncef Garouani
moncef.garouani@irit.fr

Mourad Bouneffa
mourad.bouneffa@univ-littoral.fr

1 UR 4491, Laboratoire d'Informatique Signal et Image de la Côte d'Opale, Univ. Littoral Côte d'Opale, 62100 Calais, France

2 IRIT- (CNRS/UMR 5505), Université de Toulouse Capitole, 31000 Toulouse, France

Published online: 19 January 2024

familiarize themselves with its behaviors regarding the HPs space before tuning the algorithm for a new task, iteratively. This approach can still outperform contemporary BO methods [1, 10]. By following this methodology, we argue that it is more advantageous to perform transfer learning for global black-box optimization in the framework of Bayesian Optimization to retain the proven generalization capabilities of its underlying surrogate model.

To pursue this goal, we leverage the power of meta-learning approaches [1], which initially borrow the best performing configurations of hyperparameters from the similar tasks [10–12]. These methods allow us to use the best-performing configurations of hyperparameters from similar tasks and integrate this knowledge into the acquisition function, which guides the search process toward finding high-quality solutions. By doing so, meta-learning approaches can often outperform traditional Bayesian optimization methods [4].

The Meta-learning or learning to learn is a sub-field of machine learning that focuses on learning the tasks characteristics, the ML algorithms performance and their inter dependencies [1]. In the context of hyperparameters optimization, it involves training a meta-model that learns to predict the best set of hyperparameters for a given task based on previous experiences with similar tasks. The meta-model is then used to guide the search process for a new task, allowing to quickly find good solutions without performing an exhaustive search of the entire hyperparameter space. The integration of meta-learning into the acquisition function of Bayesian optimization is a powerful approach that allows us to leverage the knowledge gained from previous tasks to guide the search process for a new task [13]. This meta-guidance can help to avoid the common pitfalls of BO, such as getting stuck in local optima or wasting computational resources on unpromising areas of the search space. By incorporating meta-learning, we can achieve better results with fewer computational resources, making hyperparameter optimization more efficient and effective [13].

The rest of the paper is structured as follows : in Sect. 2, the problem of hyperparameters optimization is formally defined and the theoretical background on Bayesian optimization is provided, along with an introduction to the functional Analysis of Variance for assessing ML hyperparameters importance. Section 3 covers details of the hyperparameters importance analysis, including the used datasets, algorithms, hyperparameters, and the functional Analysis of Variance. The proposed approach, Meta-Guided Bayesian Optimization, for speeding up the hyperparameters optimization process is presented in Sect. 4. Section 5 contains a comprehensive set of experiments involving 8 machine learning algorithms, demonstrating that MGBO outperforms state-of-the-art BO method by identifying good hyperparameters settings 2 to 3 times faster. Finally, Sect. 6 concludes the paper and provides a roadmap for future research.

2 Theoretical background and related work

In this section, we explore the theoretical foundation and two closely related research domains : (1) the Bayesian optimization for ML hyperparameters optimization, and (2) the use of functional analysis of variance to evaluate the significance of hyperparameters. Additionally, we explore the shortcomings of classical Bayesian optimization to motivate the proposed approach.

2.1 Hyperparameters optimization

The task of selecting an algorithm or a family of algorithms that can perform better on a given combination of datasets and evaluation metrics is a crucial task in machine learning [11, 14–17]. Machine learning algorithms typically have two types of parameters : ordinary parameters that the model learns and optimizes automatically during the learning phase based on the regular behavior of the model, and hyperparameters, which are set manually before training the model and these can be categorical, continuous, or conditional (see Table 1). Selecting appropriate hyperparameters can be critical to achieving optimal model performance.This process, known as hyperparameter tuning, involves finding the optimal values for these hyperparameters that result in the best performance of the model.

From a theoretical standpoint, in order to select the optimal values for hyperparameters (HPs), it is necessary to conduct a thorough exploration of all possible combinations of HPs. However, the number and variety of HPs make this task extremely challenging and potentially impractical. The risk of combinatorial explosion increases with the number of HPs and their possible values. This makes it challenging to identify the best possible combination of HPs to maximize the performance of the model. As a result, finding the optimal HPs requires a careful balance between search efficiency and accuracy. There are many techniques and tools available to help automate the process of HP selection, including grid search, random search, Bayesian optimization, and genetic algorithms [18]. Nevertheless, the challenge remains to find the most efficient and effective approach to select the optimal HPs for a given model [19].

To clarify further, let us define $\mathcal{H}$ as a population of hyperparameters configurations for a machine learning algorithms A, and $A(H)$ as the model induced by the ML algorithms A with an HPs configuration $H \in \mathcal{H}$. In traditional machine learning settings, the problem of minimizing the loss function $\mathcal{L}(A(H), \mathcal{D})$ assessing the predictive performance of $A(H)$ on a given dataset $\mathcal{D}$ is treated as a *black-box* problem (Eq. 1).

Table 1 Configuration space of some classification algorithms

ML algorithm	Nb. of hyperparameters	Nb. of ordinary parameters
Decision tree	3	1
SVM	5	2
Logistic regression	6	4
Random forest	4	2

This means that the only way to interact with the objective function $\mathcal{L}$ is by evaluating it for different hyperparameters configurations $H \in \mathcal{H}$. However, since individual evaluations of $A(H)$ on real-world datasets can take days or even weeks, the number of possible evaluations is limited, which can affect the quality of the best-found HPs configurations [5, 6].

$$\mathcal{L}(A(H), \mathcal{D}) \equiv \{\mathcal{L}(A(H), \mathcal{D}) \mid \exists H \in \mathcal{H} \sqsubseteq \bot\} \tag{1}$$

Over the past decade, a range of hyperparameters tuning techniques have been applied to optimize machine learning algorithms [10, 12, 20, 21]. Some of these techniques involve iteratively building a population $\mathcal{H}$ of hyperparameters settings and computing the loss function $\mathcal{L}(A(H), \mathcal{D})$ for each configuration of an algorithm A on a dataset $\mathcal{D}$. This allows for simultaneous exploration of different regions of the hyperparameter search space. There are various population-based HPs tuning strategies, each with a unique approach to evolving $\mathcal{H}$ at each iteration [4]. Our objective in this context is to determine the optimal HPs configuration H^+ that maximizes the loss function of a given ML algorithm A over a dataset $\mathcal{D}$ within the hyperparameters space $\mathcal{H}$, as follows:

$$\mathcal{L}(A(H^+), \mathcal{D}) \equiv \{\mathcal{L}(A(H^+), \mathcal{D}) \mid \exists H^+ \in \mathcal{L}(A(H), \mathcal{D}) \sqcap H \in \mathcal{H})\} \tag{2}$$

Similarly, the hyperparameters configuration with best configuration H^- that minimizes the loss function of a machine learning algorithm A over a dataset $\mathcal{D}$ given its hyperparameters space $\mathcal{H}$, as follows:

$$\mathcal{L}(A(H^-), \mathcal{D}) \equiv \{\mathcal{L}(A(H^-), \mathcal{D}) \mid \exists H^- \in \mathcal{L}(A(H), \mathcal{D}) \sqcap H \in \mathcal{H})\} \tag{3}$$

To address this challenge, the most simple, and often used, are Grid Search and Random Search approaches. The former is more suitable for low dimensional problems, such as, when there are few HPs to be configured. For more complex scenarios, GS is unable to explore finer promising regions due to the large hyperspace [22]. The later is able to explore any possible solution of the hyperspace, but lacks an informed search, leading to high computational costs [2]. Meta-heuristics, such as Genetic Algorithms [23], Estimation of Distribution Algorithms [19], and Sequential Model-based Optimization [4], have been explored for hyperparameters tuning due to their probabilistic nature and ability to perform informed searches. However, these techniques may require a large number of candidate configurations to be evaluated, leading to high computational costs and runtime to convergence; thus, further research is needed to eliminate the iterative evaluation of the function on less performing configurations to improve the efficiency of ML hyperparameters tuning methods [24].

2.2 Bayesian optimization

Bayesian optimization [25], also known as sequential model-based optimization, is an approach for optimizing black-box functions, such as: $f : X \rightarrow \mathbb{R}$ that are expensive to evaluate over a given input space $\mathcal{X}$. Bayesian Optimization approximates H^+ / H^- by iterative fittings of a so called *surrogate model*, henceforth denoted by Ψ, on the set of observed values denoted as Γ, of loss function $\mathcal{L}$ depending on the previous function values represented as $y_1, y_2, \ldots, y_n$ with respect to $H_1, H_2, \ldots, H_n$. The set of observed values can be formally shown as follows:

$$\Gamma \equiv \{(H_1, \mathcal{L}(A(H_1), \mathcal{D})), \ldots, (H_n, \mathcal{L}(A(H_n), \mathcal{D}))\} \tag{4}$$

The Bayesian optimization algorithm requires two building blocks, given as follows:

- Bayesian prior that prescribes a prior belief over the possible objective functions, and
- Bayesian posterior that provides a mechanism to sequentially update the belief by learning from new observations.

Bayesian optimization achieves this objective by building a Bayesian posterior on $\mathcal{L}$ based on the set of evaluated hyperparameters. At each iteration, a new configuration is selected and evaluated based on the Bayesian posterior. The later is updated to include the new HP (H_{t+1}), using a probabilistic surrogate model conditioned on the current optimization history H_t. Typically, a Gaussian Process (GP) is employed as a surrogate model in which case the resulting posterior belief

about $f(x)$ [1] follows a Gaussian distribution with mean $\mu_t(x)$ and variance $\sigma_t^2(x)$, as follows:

$$\mu_t(x) = \mathbb{E}\{f(x)|H_t \in \mathcal{H}\} \tag{5}$$
$$\sigma_t^2(x) = \mathbb{V}\{f(x)|H_t \in \mathcal{H}\} \tag{6}$$

The points explored by the Bayesian optimization algorithm are dictated by the *Acquisition Function* (AF), which assigns a utility value to each $H \in \mathcal{H}$. The utility value is determined by balancing the predicted value and uncertainty of the prediction for each H. This iterative process is repeated until a predefined budget of trials is reached, or the cross-validation performance reaches a satisfactory level. Algorithm 1 provides a representation of this process.

The effectiveness of the resulting optimizer is largely based upon carefully designing the acquisition function [20]. In this work, we used the Expected Improvement (EI) [26] as the acquisition function which quantifies the expected improvement over the best function value found so far (Eq. (7)). Alternatives to EI would be Probability of Improvement [27], Upper-Confidence Bounds [28], entropy-based methods [29], and knowledge gradient [30]. Each of these methods has its own advantages and limitations, making the choice of the acquisition function dependent on the optimization problem at hand. For instance, Probability of Improvement favors exploitation of regions with higher probabilities of finding the global maximum, while Upper-Confidence Bounds emphasizes exploration of regions with high uncertainty. Entropy-based methods prioritize the acquisition of information about the unknown function, while knowledge gradient seeks to maximize the information gain over the next set of function evaluations.

$$EI_t(x) \equiv \int_{-\infty}^{+\infty} max(y^+ - y), 0)p(y|x)\,\mathrm{d}y \tag{7}$$

where the incumbent loss value, denoted by y^+, represents the best Root-Mean-Square Error (RMSE) value found so far. The probability density function $p(y|x)$ models the uncertainty of the objective function at point x and can be represented by a probabilistic model such as a Gaussian Process (GP). The term $max(y^+ - y, 0)$ ensures that the EI function only considers improvements over the current best observation.

Therefore, the choice of the adequate acquisition function plays a crucial role in Bayesian optimization, as it determines how the optimizer balances exploration and exploitation to converge toward the optimal solution (hyperparameters configuration). In our study, we found that the Expected improvement was the most appropriate acquisition function for our optimization problem, leading to improved performance compared to other methods we tested.

2.3 Functional ANOVA for assessing ML hyperparameters importance

Functional ANOVA, or fANOVA for short, is a statistical technique used for assessing the importance of hyperparameters in machine learning algorithms [31]. It is particularly useful for hyperparameters selection and tuning as it provides insights into the relative importance of different hyperparameters in determining the performance of a given algorithm [13]. The basic idea behind functional ANOVA is to decompose the variation in the performance of the model into contributions from individual hyperparameters, as well as their interactions. This allows to identify which hyperparameters are most important for achieving high performance, and to understand how they interact with each other [31].

In fANOVA, the hyperparameters space is divided into subsets, each consisting of a single hyperparameter. The effect of each hyperparameter on the performance of the algorithm is then estimated by constructing a model that describes the relationship between the hyperparameter and the performance metric [32]. This model is constructed by fitting a function to the observed data points and decomposing it into its constituent parts, each representing the effect of a single hyperparameter. By analyzing the resulting decomposition, it is possible to determine which hyperparameters have the greatest impact on the performance of the algorithm [31]. This information can then be used to guide the hyperparameters optimization process, focusing on the most important hyperparameters to achieve the best possible performance [13].

The functional ANOVA can be applied given:

- An algorithm $A \in \mathcal{A}$ with n hyperparameters in the hyperparameters space $\mathcal{H}$.
- A large number of datasets $\mathcal{D}_1, \ldots, \mathcal{D}_m$, with m being the number of datasets (in our study, $m = 400$).
- For each of the datasets, a set of empirical performance measurements $\langle H_n, y_i \rangle_{i=1}^k$ for different hyperparameter settings $H_n \in \mathcal{H}$, where y_i is the performance of the algorithm A measured by the considered performance measures (i.e., Accuracy, Precision, Recall, F1 score).
- The *marginal performance* $\hat{a}_n(H_n)$ is defined to be the average performance of the algorithm A for all complete configurations H_n that have in its common HPs space $\mathcal{H}$.

First we collect performance data $\langle H_n, y_i \rangle_{i=1}^k$ for each of the considered algorithms with $k = 1000$ different HPs configurations. Next, we fit a random forest model to the performance data and then use the functional ANOVA to decompose the variance in performance of the random forest

[1] For the sake of simplicity, we denote H as x and $\mathcal{L}(A(H_n), \mathcal{D})$ as $f(x)$ or y.

Algorithm 1 Pseudo-code of the Sequential Model-based Optimization.

Input: Hyperparameters space $\mathcal{H}$, observation history Γ, number of iterations T, acquisition function AF, surrogate model Ψ.
Output: Best hyperparameters configuration found.
$\mathcal{L}^+ \leftarrow \varnothing$ ▷ the best value found so far
for `t =1 to T` **do**
  Fit Ψ on Γ
  $H \leftarrow \underset{x \in \mathcal{X}}{\text{argmax}}\, AF\left((\Psi(H_t), \mathcal{L}^+\right)$
  Evaluate f(H) ▷ Expensive step
  $\Gamma \leftarrow \Gamma \cup \{(H, f(H))\}$
  If $f(H) < \mathcal{L}^+$ **Then**
    $H^+, \mathcal{L}^+ \leftarrow H, f(H)$
return H^+

$\hat{y} : H_1 \times H_2 \ldots H_n \rightarrow \mathbb{R}$ into additive components that depends on subsets of the hyperparameters H_n:

$$\hat{y}(H) = \sum_{u \in n} \hat{f}_u(H_u) \tag{8}$$

where the component $\hat{f}_u(H_u)$ is defined as follows:

$$\hat{f}_u(H_u) = \begin{cases} \hat{f}_\emptyset & if u = \emptyset \\ \hat{a}_u\,(H_u) - \sum_{w \in u} \hat{f}_w(H_w) & \text{otherwise} \end{cases} \tag{9}$$

where $\hat{f}_\emptyset$ is the mean value of the function $\hat{y}$ over its domain. The unary function $\hat{f}_j(H_j)$ captures the importance of hyperparameter H_j average over all possible values for the rest of the hyperparameters, while $\hat{f}_u(H_u)$ captures the interaction effects between all hyperparameters in $\mathcal{H}$. Functional ANOVA decomposes the variance $\mathbb{V}$ in the $\hat{y}$ into the contributions $\mathbb{V}_u$ of all possible subsets of hyperparameters H_u of the algorithm $\mathcal{A}$.

$$\mathbb{V} = \sum_{u \in n} \mathbb{V}_u, \quad Where \quad \mathbb{V}_u = \frac{1}{||H_u||} \int \hat{f}_u(H_u)^2 \mathrm{d}H_u \tag{10}$$

The importance of an hyperparameter or a set of hyperparameters is captured by the fraction of the variance of the responsibility of the hyperparameter, or the set of hyperparameters. *The higher the fraction, the more important the hyperparameter or the set of hyperparameters are to the model*. Thus, such a hyperparameter should be tuned further in order to achieve a good performance.

3 Hyperparameters importance of ML algorithms across datasets

Our study involved conducting fANOVA on 400 datasets using 08 common classifiers for supervised learning. We examine *Support Vector Machine, Random Forest, Extra Trees, Adaboost, Decision Trees, Logistic Regression, SGD,* and *Gradient Boosting*. We used scikit-learn[2] to implement these classifiers and established the hyperparameters and ranges to be used in the study, which are comprehensively detailed in Table 2. In order to standardize the datasets, we applied consistent data preprocessing steps across all the training datasets. Missing values were imputed using the mode for categorical features. For numerical features, the imputation strategy was one of the hyperparameters. Additionally, categorical hyperparameters were one-hot-encoded, and constant features were removed to ensure uniformity in our analysis.

For each of the 400 datasets, previously studied in [1, 18, 33], we collected performance data for the 08 algorithms across the 400 datasets by running at least 1000 different hyperparameters configurations for each algorithm over each dataset on a powerful compute cluster.[3] This running process resulted in a knowledge base (KB) of 04 million model runs. To ensure statistical validity, we made sure that each dataset had at least 8000 runs to make functional ANOVA's model reliable enough for accessing hyperparameters importance of ML algorithms across datasets. An overview on the KB construction process is shown in algorithm 2. Detailed description of the used datasets can be found in [18].

For each dataset, we use functional ANOVA to decompose the variance in the performance of each ML classifier and analyze the contribution of each hyperparameter and their interactions. By aggregating the variance contributions over all datasets, we were able to identify the hyperparameters that have a more significant impact on performance. To provide a visual representation of this analysis, Fig. 1 presents violin plots of the distribution of importance for some algorithms hyperparameters.

By examining the plots, we can identify which hyperparameters have the greatest impact on the performance metric, as they will have a higher median importance value. This information is used to guide the selection and tuning of hyper-

[2] https://scikit-learn.org/.

[3] https://www-calculco.univ-littoral.fr/.

Table 2 Hyperparameters tuned in the experiments

Algorithm	Hyperparameter	Values
SVM	Degree	[2, 3]
	complexity (or: 'C')	[$1e^{-10}$, 500] (log-scale)
	coef0	[0., 10]
	gamma	[$1e^{-3}$, 1.01] (log-scale)
	Kernel	'poly', 'rbf'
Random Forest & Extra Trees	Min_samples_leaf	[1, 20]
	Min_samples_split	[2, 20]
	bootstrap	{true, false}
	imputation	{mean, median, mode}
	Max_features	[0.1, 0.9]
	split criterion	{entropy, gini}
Adaboost	algorithm	{SAMME, SAMME.R}
	learning rate	[0.01, 2.0] (log-scale)
	Max_depth	[1, 11]
	N_estimators	[50, 501]
Decision Trees	criterion	{'entropy', 'gini'}
	max features	[0.1, 0.9]
	min_samples_leaf	[1, 21]
	Min_samples_split	[2, 21]
Logistic Regression	C	[$1e^{-10}$, 10.] (log-scale)
	penalty	{'l2', 'l1'}
	Fit_intercept	{True, False}
SGD	Power_t	[0., 5.]
	eta0	[0., 5.]
	learning rate	{'constant', 'optimal', 'invscaling'}
	loss	{'hinge', 'perceptron', 'log'}
	penalty	{'l2', 'l1', 'elasticnet'}
Gradient Boosting	Learning_rate	[0.01, 1]
	criterion	{'friedman_mse', 'mse' }
	N_estimators	[50, 501]
	max depth	[1, 11]
	Min_samples_split	[2, 21]

Algorithm 2 The knowledge base construction process.

```
Input: ClassAlgs[..],                                  ▷ available classification algorithms
       HPSpace[..],                                    ▷ set of HPs configurations to be applied
       PerfMeasures[..]                                ▷ set of performance measures to acquire
Output: KB[#measure][#metadata]                        ▷ meta-knowledge base
function CreateMetaKB(datasets[ ])
    metadata[ ] = ∅
    for each measure in PerfMeasures do
        for each dataset D in datasets do
            DMetaFeatures = ComputeMetaFeatures(D);
            for each algorithm A in ClassAlgs do
                for each hyperparameters_configuration H in HPSpace do
                    DPerfMeasure = GetPerformanceWith5FoldCV(A, H, D);
                    metadata[ ] ← DMetaFeatures ∪ DPerfMeasure;
        meta_ds[measure] ← metadata[ ];
```

parameters in the following experiments, leading to improved overall performance.

The x-axis displays the hyperparameter being studied, while each data point corresponds to the $\mathbb{V}_i/\mathbb{V}$ value asso-

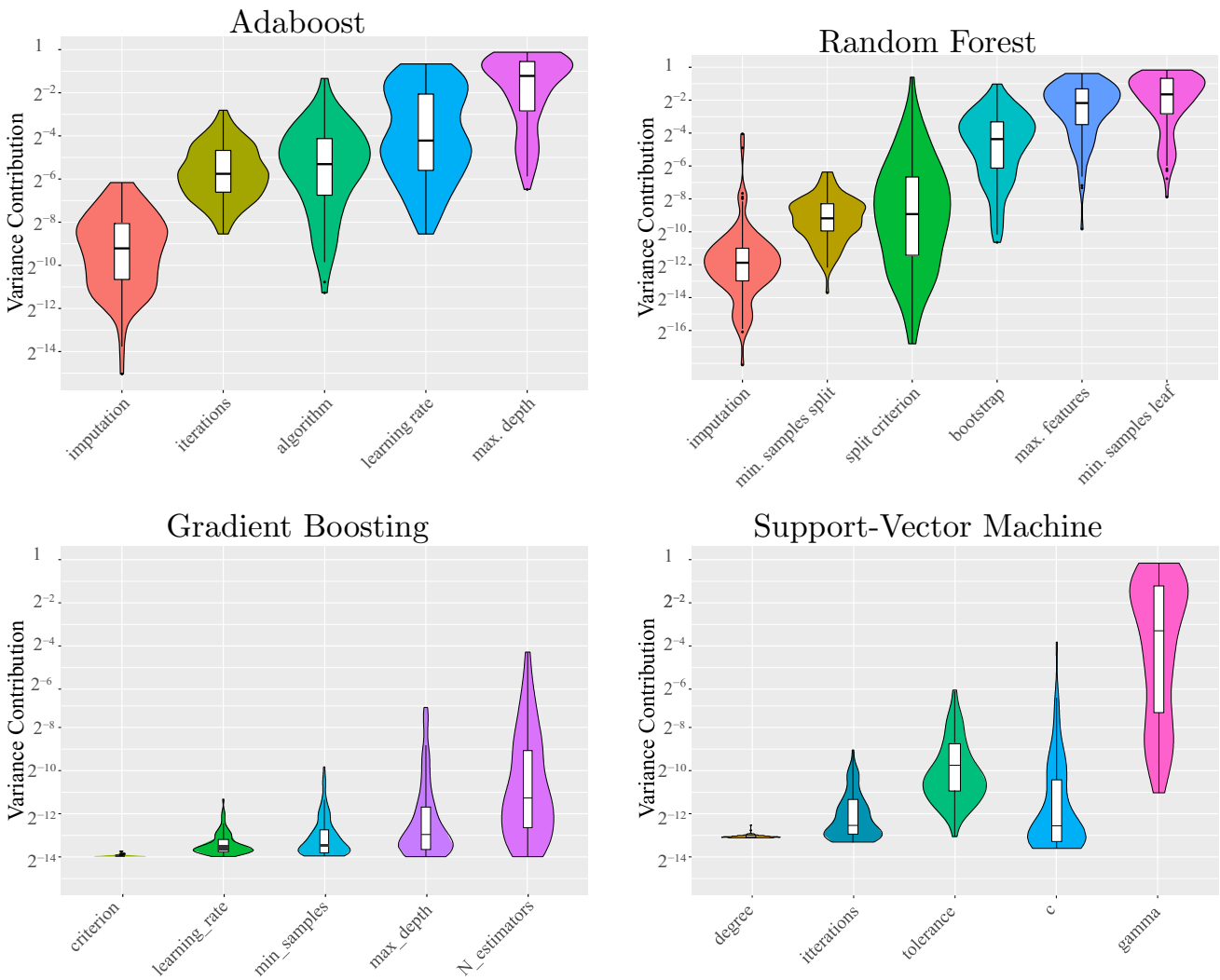

Fig. 1 Marginal contribution per 400 datasets for analyzed classifiers

ciated with that hyperparameter (eq. 10). A higher value indicates that this hyperparameter contributes significantly to the variance and could result in substantial accuracy loss if not appropriately configured. Figure 1 illustrates that, for the Adaboost classifier, the hyperparameters *maximal depth* and *learning rate* were the primary HPs influencing the classifier performance across nearly all datasets, while the *gamma* and *c* were the more significant ones for the SVM classifier.

The findings in this analysis do not suggest that tuning only the mentioned hyperparameters is sufficient. On the contrary, when there is sufficient available budget, it is still recommended to tune all hyperparameters. However, as demonstrated in a previous study [32], concentrating on adjusting the most important hyperparameters results in more rapid convergence. Therefore, in situations where the tuning budget is limited, it may be advisable to prioritize tuning the most important hyperparameters. To test this hypothesis, a meta-guided Bayesian optimization is proposed, where the tuning procedure initially concentrates on the most significant hyperparameters and gradually incorporates less important ones.

4 Meta-guided Bayesian optimization

In this section, we introduce the Meta-guided Bayesian Optimization (MGBO) approach that we propose. The MGBO algorithm integrates meta-learning into the Bayesian optimization algorithm to improve the efficiency and effectiveness of the hyperparameter tuning process. Specifically, MGBO incorporate the fANOVA to captures the importance of the hyperparameters of different machine learning algorithms. This meta-guiding model is used to guide the search for optimal hyperparameters during the Bayesian optimization process. Figure 2 gives the global architecture of the proposed meta-guided Bayesian optimization workflow.

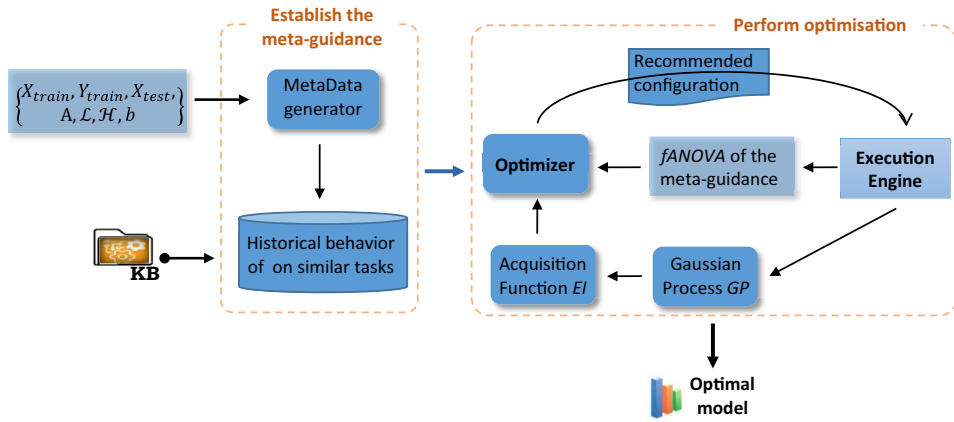

Fig. 2 Workflow of the Meta-Guided Bayesian Optimization (MGBO)

The MGBO approach proceeds in two stages. In the first stage, a meta-learning [1] process is conducted on the set of the 400 datasets to capture the relationships between the datasets characteristics and the optimal hyperparameters for various machine learning models to get the optimal ML algorithms to start with [1]. In the second stage, the fANOVA of the selected algorithm is used to guide the Bayesian optimization process for a new dataset by suggesting a set of initial hyperparameters that are likely to perform well on the given dataset. The Bayesian optimization process then iteratively refines the suggested hyperparameters by evaluating their performance on the given dataset until the optimal hyperparameters values are found.

By leveraging the fANOVA to guide the search process, MGBO reduces the number of hyperparameter configurations that need to be evaluated on the target dataset, leading to significant savings in time and computational resources. We first describe the Bayesian Optimization surrogate model used in our framework before outlining the process of selecting the next probed hyperparameters configuration.

Gaussian Process [34] is a popular choice because of its salient features such as support for noisy observations and ability to use gradient-based methods [35]. The standard GP in Bayesian optimization directly models $p(y|x)$ to estimate the expected improvement (Eq. 7). However,technically, the model does not parameterize $p(y)$, since it is computed based on the observed data points, which are heavily biased toward low values due to the optimization process. In this paper, following the Tree-structured Parzen Estimator introduced by Bergstra et al. [14], we propose the formal modeling of $p(x|y)$ and $p(y)$ instead. This is done by constructing two parametric densities, $g(x)$ and $b(x)$, which are computed using the observations with function value below and above a given threshold, respectively. The separating threshold ν is defined as a quantile (i.e., mean) of the modeled function values after observing the hyperparameters space $\mathcal{H} : \nu = mean\{f(x_i), 1 \leq i \leq n\}$. We use the densities $g(x)$ and $b(x)$ to define $p(x|y)$ as:

$$p(x|y) = g(x)I(y < \nu) + b(x)(1 - I(y < \nu)) \tag{11}$$

where:

$$I = \begin{cases} 1 \text{ if } importance(x) < importance(x_{t+1}) \\ refer\ to\ section\ 2.3 \\ 0 \text{otherwise} \end{cases} \tag{12}$$

We adopt the EI formulation used in [14] by replacing their Adaptive Parzen Estimators with our pseudo-posterior from Eq. 11, i.e.:

$$\begin{aligned} EI_y(x) &\equiv \int_{-\infty}^{+\infty} max(y^+ - y), 0)p(y|x)\,\mathrm{d}y \\ &= \int_{-\infty}^{y^+} (y^+ - y), 0)\frac{p(x|y)p(y)}{p(x)}\,\mathrm{d}y \end{aligned} \tag{13}$$

In reference to the Tree-structured Parzen Estimator, we define configurations as *"bad"* if their observed importance is below a certain quantile of the observed function values. As γ is the correlation objective values $\{y^{(i)}\}$ therefore, $p(x)$ can be represented as follows:

$$\begin{aligned} p(x) &= \int_{\mathbb{R}} p(x|y)p(y)\,\mathrm{d}y \\ &= \gamma g(x) + (1 - \gamma)b(x) \end{aligned} \tag{14}$$

moreover,

$$\begin{aligned} \int_{-\infty}^{y^+} (y^+ - y)p(x|y)p(y)\,\mathrm{d}y &= g(x)\int_{-\infty}^{y^+} (y^+ - y)p(y)\,\mathrm{d}y \\ &= \gamma y^+ g(x) - g(x)\int_{-\infty}^{y^+} yp(y)\,\mathrm{d}y \end{aligned} \tag{15}$$

$$EI_{y^+}(x) = \frac{\gamma y^+ g(x) - g(x)\int_{-\infty}^{y^+} y p(y)\,\mathrm{d}y}{\gamma g(x) + (1-\gamma) b(x)} \\ \alpha\left(\frac{1}{\gamma} + \frac{g(x)}{b(x)}(1-\gamma)\right) \tag{16}$$

The expression $EI_{y^+}(x)$ shows that in order to maximize improvement, we would like HPs configurations H, denoted as x, with high probability above $g(x)$ and under $b(x)$. This formulation makes it easy to draw many candidates according to γ and evaluate them according to $g(x)/b(x)$. On each iteration, we launch the operation argmax that seeks if the algorithm returns the candidate H^* (either H^- or H^+ depending respectively on the minimization or maximization of loss function) with the greatest EI, and, thus:

$$\underset{H\in\mathcal{H}}{\operatorname{argmax}}\, EI_{H^*}(H) = \underset{H\in\mathcal{H}}{\operatorname{argmax}}\, \frac{g(H)}{b(H)} \tag{17}$$

Typically, the minimal set of inputs to the acquisition functions in BO is given by the GP posterior prediction μ_t and σ_t. To perform the guidance, the AF has to be able to identify relevant structure shared by the objective functions in $\mathcal{L}$. In our setting, this is achieved via extending this basic set of inputs by additional features which enable the AF to evaluate sample locations. Therefore, in addition to the mean μ_t and variance σ_t at potential sample locations, the AF also receives the input importance given by the fANOVA.

It might be useful to note that the Bayesian optimization is an efficient approach for finding the optimal solution to the time-bounded optimization problem from Eq. (3), however, in its vanilla form new problems are tackled from scratch. A better alternative is to warm start BO with ML pipelines that are expected to work well, as done in the k-nearest datasets (KND) approach [1, 36]. Later on, the analysis of the candidate algorithm(s) behavior on the KND is used to bootstrap the optimizer as a meta-guidance.

5 Evaluation

Within this section, we will discuss the assessment methodology of the proposed approach for guiding the Bayesian optimization of ML algorithms. We will outline the steps taken to evaluate the effectiveness of the proposed approach and provide a detailed explanation of the metrics used to measure its performance. Furthermore, we will discuss the experimental setup and the datasets used to carry out the evaluation.

5.1 Experimental setup

We empirically evaluate the proposed MGBO approach to optimize 08 relevant Scikit-Learn machine learning algorithms. These algorithms along with their exhaustive list of the optimized hyperparameters and the search space are given in Table 2. The HPs search space is reduced to keep the computations manageable and the results interpretable.

Table 3 List of the datasets used for the experiments; the names refer to the names on the OpenML platform

Dataset	Number of instances	Number of classes
splice	3190	3
numerai28.6	96320	2
haberman	306	2
eucalyptus	736	2
abalone	4177	3
scene	2407	2
phoneme	5404	2
bank8FM	8192	2

For our experiments, we used the 08 datasets, listed in Table 3, from the OpenML[4] platform. Validation performance for Bayesian Optimization is then computed by a stratified 10-fold cross-validation fashion on the training dataset.

5.2 Experimental results

For our empirical evaluation of MGBO, we compared it to the classical Bayesian Optimization using the *Expected Improvement* as an acquisition function. For each dataset, we tracked wall clock time. Figure 3 shows the improvement obtained by our meta-guided optimization approach.

The results presented in Fig. 3 demonstrate that the meta-guided Bayesian optimization with prior guidance outperforms classical BO on all benchmark datasets. The use of prior guidance allows MGBO to achieve more efficient results by leveraging information from previously evaluated data points to guide the search toward promising areas of the input space. It is particularly noteworthy that MGBO exhibits a significant speedup in the early phases of the optimization process. On average, it requires only 13 iterations to achieve the same level of performance that classical BO may require 20 to 40 iterations to reach. This represents a speedup of approximately three times faster than classical BO.

The improved efficiency of MGBO is particularly advantageous in situations where the cost of evaluating the objective function is high or the optimization problem is time-sensitive. By reducing the number of iterations required to reach a satisfactory solution, MGBO can help save computational resources and reduce the time required to solve optimization problems. Overall, the results highlight the benefits of incorporating prior knowledge into the Bayesian opti-

[4] https://www.openml.org/.

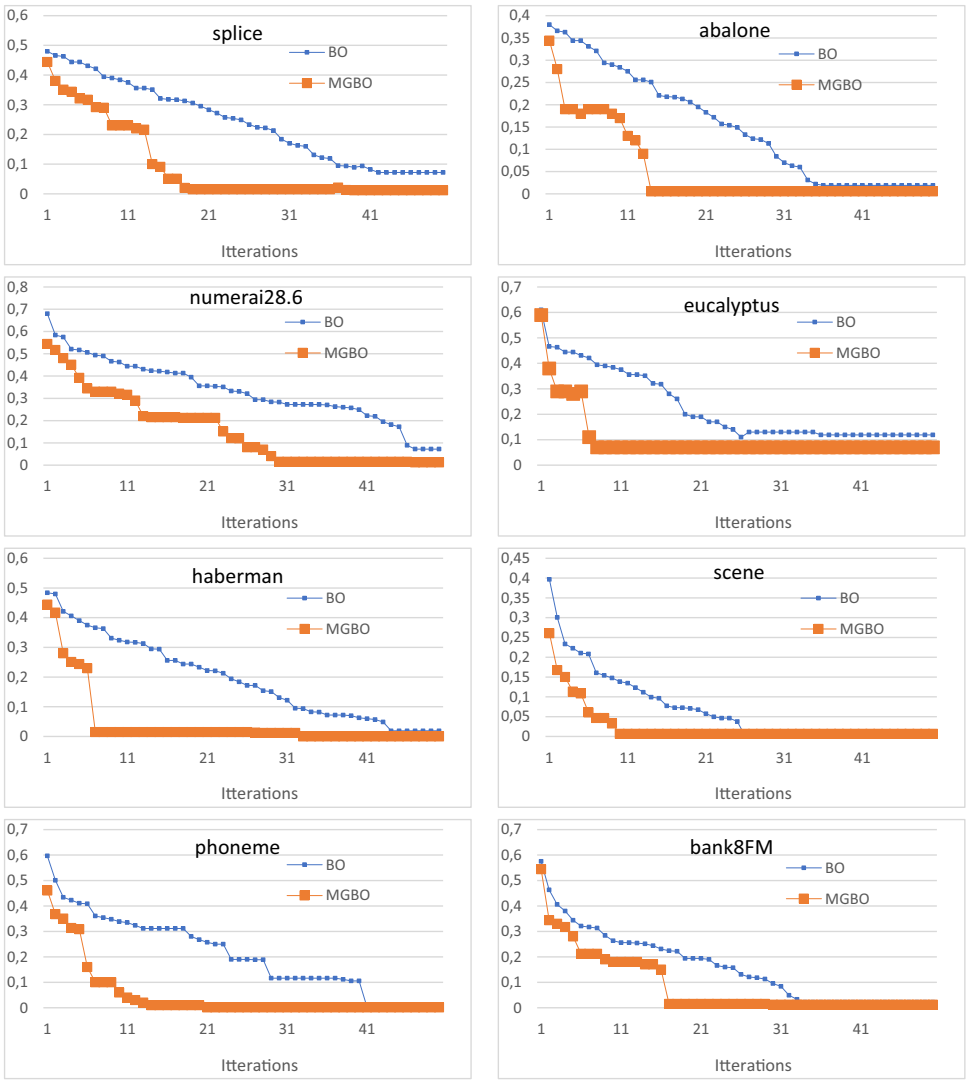

Fig. 3 Convergence of tuning policies of the traditional BO and the proposed MGBO

mization framework and suggest that MGBO is a promising approach for addressing challenging optimization problems in a more efficient manner than classical BO.

6 Conclusion

In recent years, Bayesian optimization has become a popular approach for hyperparameters tuning in machine learning. BO methods use probabilistic models to explore the hyperparameters space of a machine learning algorithm, guided by the observed performance of the algorithm on a validation set. However, Bayesian optimization can be computationally expensive, especially when dealing with high-dimensional search spaces or complex algorithms. By transferring prior knowledge about which parts of the input space can yield the best performance from previous optimization runs, we have designed a simple, yet efficient, method for improving Sequential Model-Based Bayesian Optimization of ML hyperparameters. Our approach incorporates ideas from both meta-learning and functional analysis of variance, and involves initializing and guiding BO with configurations suggested by a meta-guidance procedure. In this work, we show how black-box optimizers can be sped up by using simple meta-knowledge to guide the search space exploration. The experiments, which involved 8 ML algorithms across 8 datasets, demonstrate that our meta-guided Bayesian optimization approach can identify good HP configurations 2 to 3 times faster than the standard Bayesian optimization. In future work, we plan to study how our approach can be used to leverage even more knowledge and guidance from meta-learning. By combining the strengths of BO and meta-learning, we believe that we can further improve the efficiency and effectiveness of hyperparameters tuning in machine learning. Overall, our approach has the potential to enable faster and more accurate optimization of hyper-

parameters, leading to better machine learning models and more efficient use of computational resources.

Acknowledgements The authors would like to thank the National Center of Scientific and Technical Research (CNRST Morocco) for the financial support, and the CALCULCO computing platform, supported by SCOSI/ULCO (Service COmmun du Système d'Information de l'Université du Littoral Côte d'Opale) for the computational facilities.

References

1. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., Bourguin, G., Lewandowski, A.: Using meta-learning for automated algorithms selection and configuration: an experimental framework for industrial big data **9**(1), 57 (2022). https://doi.org/10.1186/s40537-022-00612-4
2. Muñoz, M.A., Sun, Y., Kirley, M., Halgamuge, S.K.: Algorithm selection for black-box continuous optimization problems: a survey on methods and challenges. Inf. Sci. **317**, 224–245 (2015) https://doi.org/10.1016/j.ins.2015.05.010
3. Tighineanu, P., Skubch, K., Baireuther, P., Reiss, A., Berkenkamp, F., Vinogradska, J.: Transfer learning with gaussian processes for bayesian optimization. In: Proceedings of The 25th International Conference on Artificial Intelligence and Statistics. Proceedings of Machine Learning Research, vol. 151, pp. 6152–6181 (2022). https://proceedings.mlr.press/v151/tighineanu22a.html
4. Wu, J., Chen, X.-Y., Zhang, H., Xiong, L.-D., Lei, H., Deng, S.-H.: Hyperparameter optimization for machine learning models based on Bayesian optimizationb. J. Electron. Sci. Technol. **17**(1), 26–40 (2019) https://doi.org/10.11989/JEST.1674-862X.80904120
5. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., Bourguin, G., Lewandowski, A.: Towards big industrial data mining through explainable automated machine learning. Int. J. Adv. Manuf. Technol. **120**(1), 1169–1188 (2022). https://doi.org/10.1007/s00170-022-08761-9
6. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., Bourguin, G., Lewandowski, A.: Towards meta-learning based data analytics to better assist the domain experts in industry 4.0. In: Artificial Intelligence in Data and Big Data Processing, pp. 265–277 (2022). https://doi.org/10.1007/978-3-030-97610-1_22
7. Garouani, M., Hamlich, M., Ahmad, A., Bouneffa, M., Bourguin, G., Lewandowski, A.: Toward an automatic assistance framework for the selection and configuration of machine learning based data analytics solutions in industry 4.0. In: Proceedings of the 5th International Conference on Big Data and Internet of Things, pp. 3–15 (2022). https://doi.org/10.1007/978-3-031-07969-6_1
8. Hamed, O., Hamlich, M.: Improvised multi-robot cooperation strategy for hunting a dynamic target. In: 2020 International Symposium on Advanced Electrical and Communication Technologies (ISAECT) (2020). https://doi.org/10.1109/isaect50560.2020.9523684
9. Domhan, T., Springenberg, J.T., Hutter, F.: Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. In: Proceedings of the 24th International Conference on Artificial Intelligence. IJCAI'15, pp. 3460–3468 (2015)
10. Volpp, M., Frhlich, L.P., Fischer, K., Doerr, A., Falkner, S., Hutter, F., Daniel, C.: Meta-learning acquisition functions for transfer learning in Bayesian optimization (2020) arXiv:1904.02642
11. Garouani, M., Garouani, M., Garouani, M., Ahmad, A., Bouneffa, M., Lewandowski, A., Bourguin, G., Hamlich, M.: Towards the automation of industrial data science: a meta-learning based approach, pp. 709–716 (2022). https://doi.org/10.5220/0010457107090716
12. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M.: AMLBID: an auto-explained automated machine learning tool for big industrial data. SoftwareX **17**, 100919 (2022) https://doi.org/10.1016/j.softx.2021.100919
13. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M.: Scalable meta-Bayesian based hyperparameters optimization for machine learning. In: Communications in Computer and Information Science, pp. 173–186 (2022). https://doi.org/10.1007/978-3-031-20490-6_14
14. Bergstra, J., Bardenet, R., Bengio, Y., Kégl, B.: Algorithms for hyper-parameter optimization. In: Proceedings of the 24th International Conference on Neural Information Processing Systems. NIPS'11, pp. 2546–2554, Red Hook, NY, USA (2011)
15. Garouani, M., Kharroubi, J.: Towards a new lexicon-based features vector for sentiment analysis: Application to moroccan arabic tweets. In: Advances in Information, Communication and Cybersecurity, pp. 67–76 (2022). https://doi.org/10.1007/978-3-030-91738-8_7
16. Garouani, M., Chrita, H., Kharroubi, J.: Sentiment analysis of Moroccan tweets using text mining. In: Digital Technologies and Applications. Lecture Notes in Computer Science, pp. 597–608 (2021). https://doi.org/10.1007/978-3-030-73882-2_54
17. Hamlich, M., Ramdani, M.: Applying the sac algorithm to extract the cardiologic indicators of an athlete's leve. Int. J. Intell. Inf. Syst. **5**, 23–27 (2016) https://doi.org/10.11648/j.ijiis.s.2016050301.13
18. Garouani, M.: Towards Efficient and Explainable Automated Machine Learning Pipelines Design : Application to Industry 4.0 Data. Theses, Université du Littoral Côte d'Opale; Université Hassan II (Casablanca, Maroc) (2022). https://theses.hal.science/tel-03842609
19. Padierna, L.C., Carpio, M., Rojas, A., Puga, H., Baltazar, R., Fraire, H.: Hyper-parameter tuning for support vector machines by estimation of distribution algorithms. In: Nature-Inspired Design of Hybrid Intelligent Systems, pp. 787–800 (2017). https://doi.org/10.1007/978-3-319-47054-2_53
20. Souza, A., Nardi, L., Oliveira, L.B., Olukotun, K., Lindauer, M., Hutter, F.: Bayesian optimization with a prior for the optimum. In: Machine Learning and Knowledge Discovery in Databases. Lecture Notes in Computer Science, pp. 265–296 (2021). https://doi.org/10.1007/978-3-030-86523-8_17
21. Kunjir, M.: Guided bayesian optimization to AutoTune memory-based analytics. In: 2019 IEEE 35th International Conference on Data Engineering Workshops, pp. 125–132 (2019). https://doi.org/10.1109/ICDEW.2019.00-22
22. Garouani, M., Zaysa, K.: Leveraging the automated machine learning for arabic opinion mining: a preliminary study on AutoML tools and comparison to human performance. In: Digital Technologies and Applications, pp. 163–171 (2022). https://doi.org/10.1007/978-3-031-02447-4_17
23. Priya, R., Souza, B.F., Rossi, A.L.D., Carvalho: using genetic algorithms to improve prediction of execution times of ML tasks. In: Hybrid Artificial Intelligent Systems, pp. 196–207 (2012). https://doi.org/10.1007/978-3-642-28942-2_18
24. Makarova, A., Shen, H., Perrone, V., Klein, A., Faddoul, J.B., Krause, A., Seeger, M., Archambeau, C.: Automatic termination for hyperparameter optimization (2022). https://doi.org/10.48550/arXiv.2104.08166
25. Frazier, P.I.: Bayesian optimization. In: Recent Advances in Optimization and Modeling of Contemporary Problems, pp. 255–278 (2018). https://doi.org/10.1287/educ.2018.0188
26. Mockus, J., Tiesis, V., Zilinskas, A.: The application of Bayesian methods for seeking the extremum. J. Abbr.: Towards Glob. Optim. **2**, 117–129 (2014)

27. Jones, D.R.: A taxonomy of global optimization methods based on response surfaces **21**(4), 345–383 (2001). https://doi.org/10.1023/A:1012771025575
28. Srinivas, N., Krause, A., Kakade, S., Seeger, M.: Gaussian process optimization in the bandit setting: no regret and experimental design. In: Proceedings of the 27th International Conference on International Conference on Machine Learning, pp. 1015–1022 (2010)
29. Hernández-Lobato, J.M., Hoffman, M.W., Ghahramani, Z.: Predictive entropy search for efficient global optimization of black-box functions (2014) https://doi.org/10.48550/arXiv.1406.2541
30. Wu, J., Poloczek, M., Wilson, A.G., Frazier, P.I.: Bayesian optimization with gradients (2018) https://doi.org/10.48550/arXiv.1703.04389
31. Huang, J.Z.: Functional anova models for generalized regression. J. Multivar. Anal. **67**(1), 49–71 (1998). https://doi.org/10.1006/jmva.1998.1753
32. Hutter, F., Hoos, H., Leyton-Brown, K.: An efficient approach for assessing hyperparameter importance. In: Xing, E.P., Jebara, T. (eds.) Proceedings of the 31st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 32, pp. 754–762. Bejing, China (2014)
33. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M.: Autoencoder-knn meta-model based data characterization approach for an automated selection of ai algorithms. J. Big Data **10**(14) (2023) https://doi.org/10.1186/s40537-023-00687-7
34. Rasmussen, C.E.: Gaussian processes in machine learning. Lecture Notes in Computer Science, pp. 63–71 (2004). https://doi.org/10.1007/978-3-540-28650-9_4
35. Wistuba, M., Schilling, N., Schmidt-Thieme, L.: Scalable gaussian process-based transfer surrogates for hyperparameter optimization **107**, 43–78 (2018) https://doi.org/10.1007/s10994-017-5684-y
36. Feurer, M., Eggensperger, K., Falkner, S., Lindauer, M., Hutter, F.: Auto-Sklearn 2.0: hands-free AutoML via meta-learning (2020). https://doi.org/10.48550/ARXIV.2007.04074

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.